```
$results = $xpath->query('/child::comment()');
print "Found {$results->length} comment node(s)" . PHP_EOL;
var_dump($results[0]->textContent);

$results = $xpath->query('/child::data/child::comment()');
print "Found {$results->length} comment node(s)" . PHP_EOL;
var_dump($results[0]->textContent);

Output:
Found 1 data node(s)
string(4) "data"
Found 1 comment node(s)
string(20) " A document comment "
Found 1 comment node(s)
string(20) " An element comment "
```

The heart of this example consists of three XPath queries, which serve to illustrate an important concept: the *document element*. The first query is `/child::data`, which returns a node list with a single node, which we can see from the `tagName` property is our `data` node, the root element of our XML data structure. But we used a `child` axis in our localization step. How come? Well, our `data` element has a parent: the document itself. This is further demonstrated when our `/child::comment()` localization returns the comment that lies outside `data`, just below the `<?xml version ...>` declaration; the `textContent` property of the returned node is `"A document comment"`. Logically enough, when we localize with `/child::data/child::comment()`, we're asking XPath to first localize to "nodes named `data` that are children of the document node context," then another step of "all comment nodes found in this context" (the context being `data` at this point). Remember that `comment()` is a function that selects nodes that have the type of `comment`. The only comment node found inside the `data` element contains the text `"An element comment"`, which is seen in the final output.

Now let's get a bit more crafty. We can make use of the `attribute` axis to select all the attributes of all nodes in a context like this:

```
// ... $xml, $dom, and $xpath init as before
$results = $xpath->query(
    '/child::data/descendant::git/child::project/attribute::*'
);
var_dump($results);
```